



**An End-to-End ETL Pipeline for Brazilian E-Commerce Retail Analytics:
Automated Data Ingestion, Quality Assurance, and Business Intelligence
Using Apache Airflow, PostgreSQL, and Power BI**

Author

Renato Silva.
Sandra Raj Pattuvakkaran.
Sudarsh Mekkampurath Sajeev.

Module Name

Data Engineering.

Submitted To

Prof. Dr. Nor Azizah Hitam

.

Submission Date

12 of June of 2026.

1. Project Overview & Objective

This project builds a data engineering pipeline on top of the Olist Brazilian E-Commerce datasets, public at Kaggle covering roughly 100,000 real orders placed between 2016 and 2018 across nine interrelated tables. The dataset includes orders, customers, products, sellers, payments, reviews, items, category names, and geolocation data.

The overall project goal was to take nine messy CSV files and turn them into a real ETL pipeline. It meant making decisions about what to do with missing data, how to enforce relationships between tables, and how to automate the whole process so it could run without someone pressing a button every time.

Three specific objectives drove the work:

- Build a pipeline that ingests, cleans, and loads all nine tables into PostgreSQL with enforced referential integrity.
- Automate that pipeline using Apache Airflow so it runs on a schedule with monitoring and failure handling.
- Connect the cleaned database to Power BI and deliver a dashboard covering delivery performance, revenue trends, seller rankings, and customer review scores.

2. Problem Statement & Methodology

The raw Olist data is nine separate CSV files with no enforced relationships between them. On its own it cannot answer a single business question. Calculating delivery times requires joining orders to order items to products. Analysing revenue by region requires linking customers to geolocation. The data exists but it is not usable as it is.

The engineering problem is building a pipeline that cleans the source files, enforces the relationships, and loads everything into a structure that supports reliable queries. The additional constraint is that this pipeline should run automatically and handle failures with precision, not just work once.

The methodology followed a standard ETL approach with a dev/test split added to validate usability:

- Extract raw CSV files via the Kaggle API.
- Explore the data to identify quality issues.
- Transform by resolving nulls, duplicates, type mismatches, and missing foreign key references.
- Split 75/25 into development and test sets anchored on the orders table.
- Load the development set into PostgreSQL with a 3NF schema.
- Validate using Apache Airflow DAG execution and Python integrity checks.
- Test by running the same validation against the 25% test set.

The dev/test split mirrors how production pipelines are validated, the pipeline is built and tuned on one batch of data, then proven on a second batch it has never seen.

3. Data Collection & Preparation

Data was extracted programmatically using the Kaggle API. The pipeline calls the API, downloads the dataset, and unzips all nine files into a raw data folder. This was the first point where environment issues appeared: Our network uses an SSL proxy that blocked standard pip installs and API calls. The fix was adding --trusted-host flags to bypass certificate verification. Once the files were loaded into Python, the exploration phase identified four categories of data quality issues:

Table	Issue	Decision	Reason
orders	2,965 null delivery dates	Preserved	Represent undelivered orders valid data.
products	610 missing category names	Filled with 'unknown'	Category must exist for FK constraint
products	2 missing dimensions	Filled with median	Median avoids skew from outliers
reviews	87,656 missing titles	Filled with 'No title'	Most customers skip written feedback
reviews	58,247 missing messages	Filled with 'No comment'	Same reason as above
reviews	Duplicate review IDs	Keep first, drop rest	Bug in Olist's review submission system
category_translation	Missing categories (pc_gamer etc.)	Added programmatically	Required for FK integrity with products

4. ETL Pipeline & Storage Design

The pipeline follows a linear sequence: extract from Kaggle, transform in Python, load into PostgreSQL. Apache Airflow orchestrates the execution order and scheduling. The architecture diagram below shows the full flow:



Figure 1: End-to-End Pipeline Architecture

The PostgreSQL schema follows third normal form(3NF). Each piece of information lives in exactly one table, connected to others through primary and foreign keys. The schema separates reference tables such as: products, sellers, geolocation, category translation. From transaction tables: orders, customers, order items, payments, reviews. Reference tables are loaded first because transaction tables hold foreign keys that point to them.

Key schema decisions:

- Order items use a composite primary key on `order_id` and `order_item_id` together, because neither column is unique on its own because one order can contain multiple items.
- Payments uses the same composite key pattern on `order_id` and `payment_sequential`, because one order can be split across multiple payment methods.
- Geolocation has no primary key because the same zip code prefix maps to multiple lat/lng coordinate points.

The entity-relationship diagram below shows the full schema with all primary and foreign key relationships:

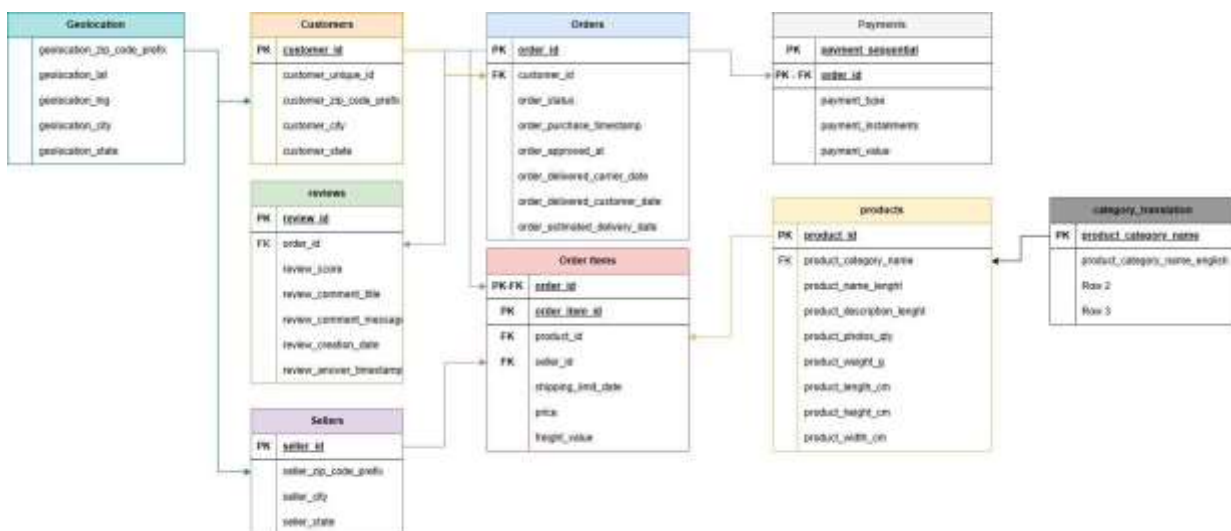


Figure 2: Entity-Relationship Diagram — olist ecommerce database

The screenshot below confirms all nine tables were successfully created and populated in PostgreSQL 18:

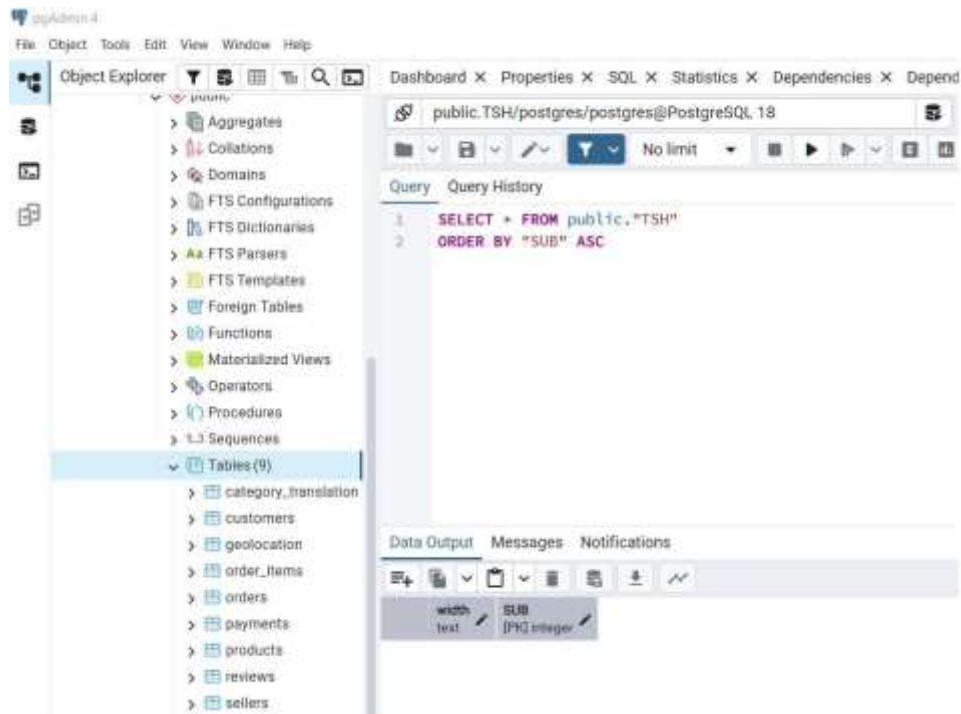


Figure 3: PostgreSQL pgAdmin showing all 9 tables in olist_ecommerce database

5. Orchestration & Quality Assurance

The pipeline is orchestrated by an Apache Airflow DAG with four tasks running in sequence: extract, clean, validate, and notify. The DAG runs on a daily schedule with one automatic retry per task before marking a failure.

The Airflow DAG interface showing the pipeline structure:

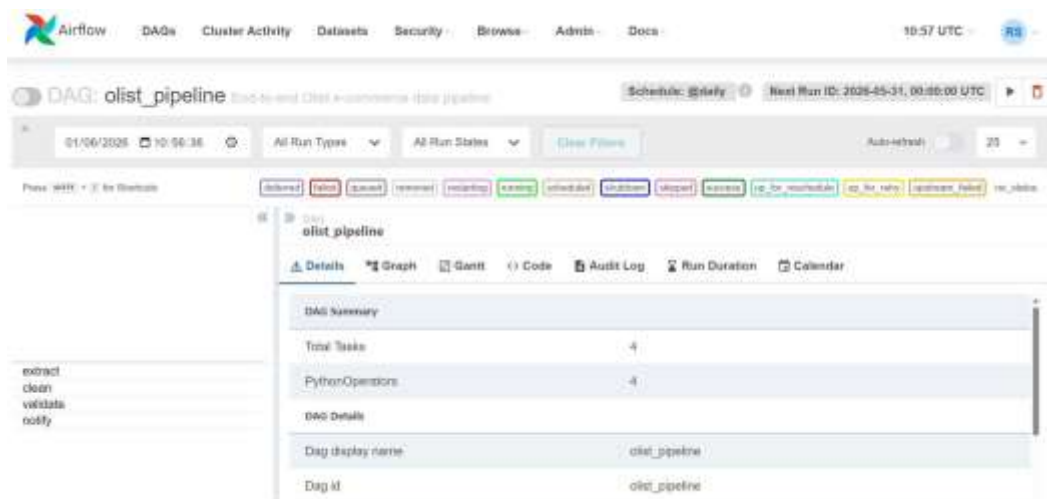


Figure 4: Airflow DAG olist_pipeline — details view showing 4 PythonOperator tasks

The screenshot below confirms all four tasks completed successfully in the most recent run:

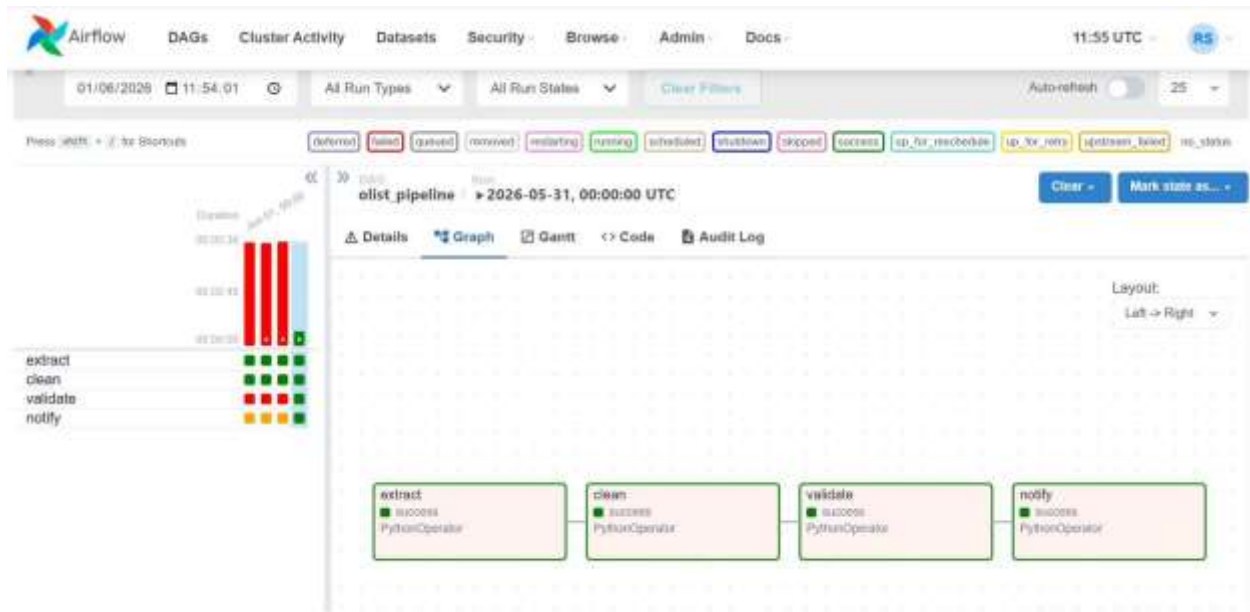


Figure 5: Airflow DAG `olist_pipeline` — all 4 tasks green (extract, clean, validate, notify)

Environment configuration in windows.

Setting up Airflow on Windows required WSL since Airflow does not run natively on Windows. Inside WSL, Python 3.14 was the system default, but Airflow 2.9.2 requires Python 3.11. A separate virtual environment was created with Python 3.11. Connecting WSL to Windows PostgreSQL required finding the Windows host IP from WSL's network configuration and whitelisting the WSL subnet in `pg_hba.conf`.

Data quality assurance covered three areas:

- Null handling was documented for each affected column with an explicit decision: fill, drop, or preserve.
- Foreign key integrity was validated in Python before any data reached PostgreSQL, checking all four key relationships across the dataset.
- The test set validation confirmed zero broken references across all four integrity checks, proving the pipeline produces clean data on unseen input.

Test set validation results:

Check	Result	Status
Order items with invalid order_id	0	PASS
Orders with invalid customer_id	0	PASS
Order items with invalid product_id	0	PASS
Order items with invalid seller_id	0	PASS
Test orders row count	24,860 (expected ~24,860)	PASS

6. Results & Analytical Insights

The development set loaded 74,581 orders across all nine tables with zero constraint violations. The test set validation confirmed zero foreign key violations across all four integrity checks.

Row Count Summary

Table	Dev Set	Test Set	Full Dataset
Orders	74,581	24,860	99,441
Customers	74,581	24,860	99,441
order_items	84,439	28,143	112,650
Payments	77,931	26,060	103,886
Reviews	73,804	24,785	~98,589
Products	32,951	32,951	32,951
Sellers	3,095	3,095	3,095
Geolocation	1,000,163	1,000,163	1,000,163
category_translation	74	74	74

Key Analytical Findings

- Sao Paulo accounts for roughly 40% of all orders. It is consistent with its position as Brazil's largest economic hub.
- The average review score is 5 out of 5, indicating generally high customer satisfaction across the platform.
- Review score distribution follows the lead of the 5-star reviews as the most common, but 1-star reviews are the third most common.

- Order items always exceed order count because one order can contain multiple products. Payments also exceed orders because some customers split payments across multiple methods.
- Not every customer leaves a review: approximately 600 orders in the dev set have no corresponding review record.

The Power BI dashboard covers three analytical tabs: Overview (total KPIs and order trends), Delivery Performance (on-time vs late analysis by state and month), and Commerce & Reviews (revenue by category, top sellers, and review distribution).

7. Challenges & Recommendations

Technical Challenges Encountered

Challenge	Root Cause	Resolution
SSL certificate errors on pip install and Kaggle API	Our network SSL proxy intercepting HTTPS connections	Added trusted-host flags to bypass certificate verification
Python 3.14 incompatible with Airflow 2.9.2	System Python version too new for Airflow requirements	Created separate Python 3.11 virtual environment inside WSL
WSL could not connect to Windows PostgreSQL	localhost in WSL points to Linux, not Windows host	Found Windows host IP via /etc/resolv.conf, whitelisted WSL subnet in pg_hba.conf
Foreign key violations on product load	Products referenced category names not in category translation table	Programmatically detected and added all missing categories before loading
Duplicate review IDs causing primary key violations	Bug in Olist review submission system, same ID reused for multiple submissions	Applied drop duplicates keeping first occurrence

Recommendations for Improvement.

- Implement UPSERT logic using PostgreSQL ON CONFLICT DO NOTHING syntax so the pipeline handles incremental batch loads without dropping and reloading the entire database.
- Move raw data storage to cloud object storage such as AWS S3 so the pipeline is not dependent on local file paths.
- Add data quality checks directly into the Airflow DAG as a dedicated task between clean and validate, automating the integrity checks currently done manually in the notebook.

- In a production environment, the SSL proxy issue would require proper certificate installation rather than bypassing verification.

8. References & Supporting Material

Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle.

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Apache Software Foundation. (2024). Apache Airflow documentation (Version 2.9.2).

<https://airflow.apache.org/docs/>

PostgreSQL Global Development Group. (2024). PostgreSQL 18 documentation.

<https://www.postgresql.org/docs/18/>

Microsoft. (2024). Power BI documentation. <https://learn.microsoft.com/en-us/power-bi/>

McKinney, W. (2022). Python for data analysis (3rd ed.). O'Reilly Media.

SQLAlchemy. (2024). SQLAlchemy documentation (Version 2.0). <https://docs.sqlalchemy.org/>